DOCUMENTO DE ARQUITECTURA

Documento de Arquitectura de Software

Sistema de Soporte Decisional para la Reducción del Desperdicio Alimentario en Bancos de Alimentos

PARA

Jenny Alexandra Múnera Serna
Asesora · Maestría en Arquitectura de Software, Universidad de La Sabana

POR

Luis Alberto Rojas Adames, Astrid Carolina Rodríguez Cristancho y Alexander Caballero García
Maestría en Arquitectura de Software · Universidad de La Sabana
Chía, Cundinamarca · 2026

Control del Documento

Versión	Fecha	Cambiado por	Comentarios
1.0	2026-10-11	Luis Alberto Rojas Adames, Astrid Carolina Rodríguez Cristancho, Alexander Caballero García	Versión inicial del documento de arquitectura.
1.1	2026-10-12	Luis Alberto Rojas Adames, Astrid Carolina Rodríguez Cristancho, Alexander Caballero García	Latido SSE cada 15 s en el patrón de SSE y en ADR-008; el ALB ya no necesita ampliar su tiempo de espera (cambio posterior a la aprobación de decisions.md).
2.0	2026-10-17	Luis Alberto Rojas Adames, Astrid Carolina Rodríguez Cristancho, Alexander Caballero García	Revalidación tras la revisión del anteproyecto: drivers, decisiones, vistas y validación actualizados.

Tabla de Contenido

Introducción 5

1. Contexto de la Solución 5

1.1 Situación Actual 5

1.2 Objetivo 5

1.3 Vista de Contexto C4 6

2. Drivers Arquitectónicos 6

2.1 Funcionalidades Principales 6

2.2 Restricciones 7

2.3 Supuestos 7

2.4 Atributos de Calidad 8

3. Decisiones de Arquitectura 12

3.1 Estilo Arquitectónico: Monolito Modular 12

3.2 Patrones 13

3.2.1 Clean Architecture 13

3.2.2 Domain-Driven Design 13

3.2.3 CQRS 14

3.2.4 Unit of Work 14

3.2.5 Comunicación entre módulos por interfaces 14

3.2.6 Análisis disparado por movimiento con recálculo periódico 14

3.2.7 Control de acceso basado en roles 14

3.2.8 Alta de usuarios por invitación 15

3.2.9 Registro de autorizaciones de tratamiento de datos 15

3.2.10 Minimización de datos personales 15

3.3 Base de Datos 16

3.4 Servicios Cloud Seleccionados 16

3.5 Stack Tecnológico 17

4. Definición de Arquitectura 18

4.1 Vista Funcional 18

4.1.1 Diagrama de contenedores 18

4.1.2 Diagrama de componentes 19

4.2 Vista de Información (Base de Datos) 20

4.3 Vista de Despliegue 22

4.4 Vista de Desarrollo (Paquetes) 22

5. ADRs 24

ADR-001: Estilo arquitectónico 25

ADR-003: Comunicación con el módulo de análisis 26

ADR-004: Registro del inventario....26
ADR-005: Reglas de negocio configurables....27
ADR-007: Plataforma de despliegue en AWS....28
ADR-010: Aplicación de FEFO en la remisión....29
ADR-011: Preparación para varios bancos....30
ADR-012: Validación de la integridad de los datos....31
ADR-013: Salidas simultáneas sobre el mismo lote....32
ADR-014: Confirmaciones sin duplicar al reintentar....32
ADR-015: Almacenamiento de los adjuntos de la recepción....33
ADR-016: Interfaz para el registro en bodega....34
ADR-017: Comunicación y transacciones entre módulos....35
6. Validación de la Arquitectura....36
6.1 Análisis por Escenario....36
6.2 Riesgos....38
6.3 Temas de Riesgo....39

Introducción

Este documento describe la arquitectura del Sistema de Soporte Decisional para el Banco Diocesano de Alimentos de Neiva, un prototipo que registra el inventario por lotes y movimientos, traza cada entrega hasta la donación o compra de la que salió y alerta el riesgo de vencimiento, para apoyar las decisiones de la coordinación del banco y reducir el desperdicio de alimentos. Cubre el contexto, los drivers arquitectónicos, las decisiones de diseño, las vistas del sistema, los registros de decisiones (ADR) y la validación de la arquitectura frente a sus escenarios de calidad.

1. Contexto de la Solución

1.1 Situación Actual

El Banco Diocesano de Alimentos de Neiva registra en papel y en hojas de Excel sus entradas y sus entregas, sin lotes ni existencias. Las entradas son donaciones y también compras propias. Por eso no puede relacionar una entrega con la entrada de la que salió. Tampoco sabe cuánto se pierde en bodega ni qué está próximo a vencer, porque el vencimiento se controla con una revisión diaria y manual que no deja registro. Además, el mismo dato se digita en varios formatos hasta armar a mano el reporte mensual para la red de bancos, que sale con errores de digitación y cuya estructura cambia. Le duele a la coordinación, que decide sin información de inventario, y al personal administrativo y de bodega, que redigita. El problema central no es solo el desperdicio. Es la falta de información confiable y trazable, que impide medirlo, anticiparlo y priorizar la distribución. El alcance es un prototipo para el banco de Neiva, que podría extenderse a otros bancos si el modelo tiene éxito. El trabajo de grado es la oportunidad de diseñar un sistema de soporte a decisiones que ataque la trazabilidad y el riesgo de vencimiento.

Actores: Personal de bodega: recibe y selecciona las donaciones y las compras; registra bajas, ajustes y salidas a compostaje.; Asistente administrativa: registra destinatarios, emite remisiones y genera reportes.; Coordinador del banco: consulta inventario, alertas y trazabilidad y decide sobre los productos en riesgo; configura catálogo, programas, kits, vida útil y reglas de riesgo; da de alta a los demás usuarios de su banco y les asigna su rol.; Administrador: tiene todos los permisos y no pertenece a ningún banco; da de alta el banco y a su coordinador, y puede operar en cualquier banco.; Destinatario: persona natural o jurídica (organización) que recibe y firma la remisión; no usa el sistema.; Donante: persona natural o jurídica que entrega la donación con su soporte; no usa el sistema.; Proveedor: persona natural o jurídica a la que el banco le compra productos para distribuirlos; entrega la factura y no usa el sistema..

Sistemas externos: EatCloud: canal por el que llegan anuncios de donación; no se integra, el anuncio se registra a mano.; Red de bancos de alimentos (¿ABACO?): recibe el reporte mensual de entregas..

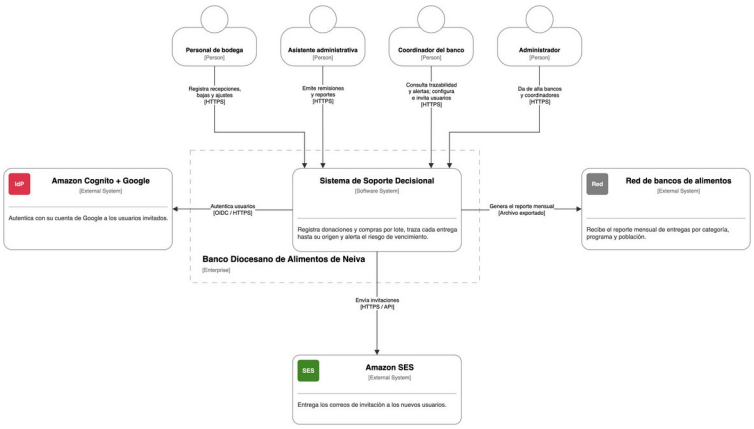
1.2 Objetivo

Reducir el desperdicio: medir las pérdidas en recepción y en bodega, e identificar a tiempo los productos en riesgo de vencerse para priorizar su distribución. Medida: kg desechados y dados de baja por mes, por donante, categoría y motivo, disponibles en el sistema.

Trazar cada producto desde su origen (donante o proveedor) hasta el destinatario. Medida: 100 % de los kg entregados vinculados a un lote y a una entrada registrada.

Dejar de redigitar registros y de armar a mano el reporte mensual. Medida: reporte mensual generado sin volver a digitar datos.

1.3 Vista de Contexto C4



2. Drivers Arquitectónicos

2.1 Funcionalidades Principales

- UC-01 Registrar recepción: El personal de bodega registra una entrada, sea donación o compra, puede adjuntar el documento que entrega el donante o la factura de la compra (opcional), y registra la selección en aprovechado y desechado; cada renglón aprovechado queda como lote con su fecha de vencimiento (de empaque, estimada o sin fecha si no es perecedero).
- UC-02 Consultar existencias y trazabilidad: El coordinador consulta las existencias por producto, lote y vencimiento, y el recorrido de una entrega hasta su donante o proveedor.
- UC-03 Emitir remisión: La asistente administrativa registra una salida a un destinatario, opcionalmente por programa y número de kits; el sistema asigna el lote de cada renglón por FEFO.
- UC-04 Alertar riesgo de vencimiento: El sistema calcula el riesgo de vencimiento de cada producto perecedero y el coordinador recibe las alertas.
- UC-05 Registrar bajas y ajustes: El personal de bodega registra bajas (vencido, dañado), salidas a compostaje y ajustes de inventario, cada uno con su motivo.
- UC-06 Generar reporte mensual: La asistente administrativa genera el reporte mensual por categoría, programa y población para la red de bancos.
- UC-07 Gestionar catálogo: El coordinador registra productos, categorías, presentaciones y la vida útil por categoría.
- UC-08 Registrar destinatario: La asistente administrativa registra a una persona natural o jurídica y, si es organización, su vinculación y población atendida.
- UC-09 Configurar programas y kits: El coordinador define los programas y la composición de los kits de los que la tienen.
- UC-10 Configurar reglas de riesgo: El coordinador ajusta los umbrales de riesgo y la ventana del ritmo de salida.
- UC-11 Gestionar bancos y coordinadores: El administrador da de alta un banco y a su coordinador.
- UC-12 Gestionar usuarios del banco: El coordinador da de alta a los demás usuarios de su banco y les asigna su rol.
- Funcionalidad primaria: UC-01 a UC-07, UC-09 y UC-10. UC-01, UC-02, UC-03 y UC-05 son lógica de negocio crítica. Ahí nacen y se consultan la trazabilidad y la medición del desperdicio, y por eso exigen lotes y movimientos en lugar de totales. UC-05 entra porque mide las pérdidas en bodega y es el camino de las excepciones a FEFO. UC-04 es el soporte a decisiones y debe poder cambiar sin afectar la operación.

UC-06 entrega datos a la red de bancos en un formato que cambia. UC-07, UC-09 y UC-10 concentran reglas que cambian con frecuencia y no deben requerir código. UC-08, UC-11 y UC-12 no son primarios porque son registros con un flujo simple. Fuera de alcance: la cartera (aporte solidario y cuentas por cobrar); la asignación de productos a destinatarios y la distribución equitativa; la importación de históricos en Excel; el modelo predictivo, que es una extensión futura; la operación con otros bancos (proyección, AS-5).

2.2 Restricciones

- CON-1: Es un prototipo de maestría limitado a los flujos críticos: entradas con lote, salidas vinculadas al lote, consulta de trazabilidad y alerta de vencimiento. Se valida con escenarios de calidad y pruebas, y no entra a producción.
- CON-2: El equipo son tres personas y el plazo es el cronograma del trabajo de grado (octubre de 2026 a febrero de 2027).
- CON-3: Como entidad sin ánimo de lucro, la operación debe tener costo bajo.
- CON-4: Se tratan datos personales de representantes, de personas naturales que reciben ayuda y de la población atendida, por lo que aplica la Ley 1581 de 2012.
- CON-5: El sistema debe generar documentos equivalentes a los formatos institucionales FT-ADM, como la remisión firmada.
- CON-6: El riesgo de vencimiento se calcula con reglas sobre días al vencimiento, existencias y ritmo de salida; el análisis predictivo se incorpora después, sin modificar el núcleo de trazabilidad.
- CON-7: La interfaz debe ser usable por el personal de bodega durante la recepción, con los términos de los formatos FT-ADM y desde celular o tablet.
- CON-8: El sistema separa un núcleo operativo de trazabilidad (entradas, salidas e inventario por lote) de una capa de soporte a la decisión, de modo que las reglas de decisión evolucionen sin modificar el registro de las operaciones.

2.3 Supuestos

- AS-1: La carga es baja: del orden de 30–90 t al mes, ~20 remisiones por día y menos de 10 usuarios a la vez. Si es falso: habría que replantear capacidad y concurrencia. Estado: Por validar con el banco.
- AS-2: La bodega tiene internet estable en horario operativo. Si es falso: la recepción se bloquea; haría falta un modo sin conexión. Estado: Por validar con el banco.
- AS-3: No existe un histórico digital de entradas con vencimiento ni de pérdidas en bodega. Si es falso: si existe, el modelo predictivo podría entrenarse desde el inicio (riesgo favorable). Estado: Por validar con el banco.
- AS-4: La panadería y el fruver llegan sin fecha de vencimiento y se estiman con la vida útil de su categoría; el aseo no vence y sale por orden de llegada. Si es falso: sin fecha ni regla no se puede calcular su riesgo; si el aseo tiene otro criterio, cambia su orden de salida. Estado: Por validar con el banco.
- AS-5: Si el modelo tiene éxito, el sistema podrá operar en otros bancos de alimentos, cada uno con sus datos, usuarios y roles aislados. Si es falso: la preparación para varios bancos sería un costo de diseño sin uso. Estado: Proyección del arquitecto; fuera del árbol de utilidad.
- AS-6: No hay integración con servicios externos (EatCloud, red de bancos); el intercambio se hace por registro manual o por archivos. Si es falso: si la red de bancos exige una API, el reporte necesitaría un adaptador de integración. Estado: Por validar con el banco.
- AS-7: Ningún destinatario pide un lote específico; basta con el producto. Si es falso: fEFO obligatorio no podría atender ese pedido. Estado: Por validar con el banco.
- AS-8: Los kits se arman al despachar a partir de una plantilla por programa, y cada remisión lleva como máximo un programa, como en el FT-ADM-006. Si es falso: si se arman con anticipación, el kit armado

sería existencia en bodega; si se mezclan programas, la remisión necesita varios grupos de kits. Estado: Por validar con el banco.

- AS-9: Además de las organizaciones, el banco entrega a personas naturales y en emergencias, y emite remisiones sin programa. Si es falso: el modelo de destinatario sería más general de lo necesario, con un costo bajo. Estado: Por validar con el banco.
- AS-10: Las compras llegan con factura de un proveedor y pasan por la misma selección y el mismo inventario por lote que las donaciones. Si es falso: si las compras no se seleccionan o llegan por otro canal, la recepción necesitaría un flujo distinto. Estado: Por validar con el banco.

2.4 Atributos de Calidad

El árbol de utilidad resume los atributos de calidad, sus refinamientos y los escenarios que los concretan, con su prioridad para el negocio y su dificultad técnica (A = alta, M = media, B = baja).

Atributo	Refinamiento	Escenario	Prioridad (N, D)
Trazabilidad	Procedencia de lo entregado	QA-01 — Origen de un producto entregado	(A, A)
Trazabilidad	Inmutabilidad del historial	QA-02 — Corrección sin borrar historia	(A, M)
Integridad de datos	Validación en la captura	QA-03 — Rechazo de registros inconsistentes	(A, M)
Fiabilidad	Aislamiento de fallos del análisis	QA-04 — Operación sin el análisis	(A, M)
Fiabilidad	Recuperabilidad sin pérdida de registros	QA-05 — Recuperación sin pérdida de registros	(A, M)
Mantenibilidad	Configurabilidad de reglas de negocio	QA-06 — Cambio de reglas por parámetros	(A, M)
Mantenibilidad	Reemplazabilidad de las reglas de decisión	QA-07 — Reemplazo de las reglas de decisión	(M, M)

ID	QA-01 — Origen de un producto entregado
Atributo de calidad	Trazabilidad › Procedencia de lo entregado
Prioridad	Negocio A · Dificultad A
Fuente del estímulo	Coordinador del banco
Estímulo	Consulta el origen de un producto entregado a un destinatario
Ambiente	Operación normal
Artefacto	Registro de lotes, movimientos y remisiones
Respuesta	Reconstruye destinatario, remisión, lote, recepción, soporte y donante o proveedor
Medida de la respuesta	Menos de 5 s y sin eslabones faltantes

ID	QA-02 — Corrección sin borrar historia
Atributo de calidad	Trazabilidad › Inmutabilidad del historial
Prioridad	Negocio A · Dificultad M
Fuente del estímulo	Personal de bodega
Estímulo	Corrige la cantidad de un lote
Ambiente	Operación normal
Artefacto	Registro de movimientos
Respuesta	La corrección queda como un movimiento nuevo con motivo y usuario, y el original se conserva
Medida de la respuesta	0 registros borrados o sobrescritos

ID	QA-03 — Rechazo de registros inconsistentes
Atributo de calidad	Integridad de datos › Validación en la captura
Prioridad	Negocio A · Dificultad M
Fuente del estímulo	Personal de bodega o asistente administrativa
Estímulo	Intenta registrar un dato incompleto o inconsistente: un lote perecedero sin fecha en una categoría sin vida útil, una salida mayor que las existencias, un producto fuera del catálogo, un vencido como aprovechado o una remisión sin destinatario
Ambiente	Operación normal, incluidas dos salidas simultáneas del mismo lote
Artefacto	Módulos de inventario y distribución
Respuesta	Rechaza el registro en el momento, indica qué corregir y no guarda nada parcial
Medida de la respuesta	0 registros inconsistentes guardados

ID	QA-04 — Operación sin el análisis
Atributo de calidad	Fiabilidad › Aislamiento de fallos del análisis
Prioridad	Negocio A · Dificultad M
Fuente del estímulo	Falla interna
Estímulo	El componente de análisis deja de responder
Ambiente	Jornada de recepción, sábados incluidos
Artefacto	Módulos de inventario y distribución
Respuesta	Recepción y remisiones siguen operando; las alertas pendientes se recalculan cuando el componente se recupera

ID	QA-04 — Operación sin el análisis
Medida de la respuesta	0 recepciones o remisiones bloqueadas por una falla del análisis

ID	QA-05 — Recuperación sin pérdida de registros
Atributo de calidad	Fiabilidad › Recuperabilidad sin pérdida de registros
Prioridad	Negocio A · Dificultad M
Fuente del estímulo	Falla interna
Estímulo	El servidor de aplicación falla mientras se confirma una recepción o una remisión
Ambiente	Operación normal
Artefacto	Transacciones de recepción y remisión en el servidor de aplicación y la base de datos
Respuesta	El servidor se reinicia sin intervención; cada operación queda completa o no queda, y reintentarla no la duplica
Medida de la respuesta	0 registros confirmados perdidos o a medias

ID	QA-06 — Cambio de reglas por parámetros
Atributo de calidad	Mantenibilidad › Configurabilidad de reglas de negocio
Prioridad	Negocio A · Dificultad M
Fuente del estímulo	Coordinador del banco
Estímulo	Cambia la composición de un kit, la vida útil de una categoría o un umbral de riesgo
Ambiente	Operación normal
Artefacto	Catálogo, programas y kits, y parámetros del análisis
Respuesta	Aplica el cambio por parámetros, sin tocar código ni desplegar; el cambio queda registrado y solo afecta lo que viene
Medida de la respuesta	Menos de 1 hora

ID	QA-07 — Reemplazo de las reglas de decisión
Atributo de calidad	Mantenibilidad › Reemplazabilidad de las reglas de decisión
Prioridad	Negocio M · Dificultad M
Fuente del estímulo	Equipo de desarrollo
Estímulo	Reemplaza las reglas de riesgo por un modelo predictivo

ID	QA-07 — Reemplazo de las reglas de decisión
Ambiente	Mantenimiento
Artefacto	Componente de análisis (capa de soporte a la decisión)
Respuesta	El cambio queda contenido en el componente de análisis; el núcleo operativo no se modifica
Medida de la respuesta	0 cambios en inventario, distribución y catálogo

3. Decisiones de Arquitectura

3.1 Estilo Arquitectónico: Monolito Modular

El sistema se despliega como una sola aplicación .NET dividida en módulos por contexto del dominio. Cada módulo expone su funcionalidad a los demás mediante interfaces, dentro del mismo proceso, sin acceder a los datos de otro módulo. Un solo despliegue mantiene bajo el esfuerzo de un equipo de tres personas (CON-2) y el costo de operación (CON-3), y basta para la carga esperada (AS-1). Los límites entre módulos dejan contenidos los cambios del análisis (QA-07) y permitirían extraer un módulo más adelante. La excepción es el módulo de análisis de vencimiento, que es la capa de soporte a la decisión (CON-8). Se despliega en un contenedor aparte, comunicado con el monolito por HTTP, para que su falla no afecte la operación (QA-04) y para poder reemplazar las reglas por un modelo predictivo, incluso en otro lenguaje (QA-07).

Módulos del sistema:

• Inventario (núcleo del dominio): recepción de donaciones y compras con su adjunto opcional, lotes con el origen de su fecha de vencimiento, movimientos inmutables, bajas, ajustes, compostaje, existencias y trazabilidad (UC-01, UC-02, UC-05).

• Distribución: programas y plantillas de kit, remisiones con número de kits y renglones ajustables con motivo, en la unidad de conteo de cada producto, asignación de lotes por FEFO obligatorio (FIFO para no perecederos) y remisiones en PDF con plantilla equivalente al FT-ADM-006 (UC-03, UC-09).

• Catálogo (soporte, del que dependen los demás módulos): productos, categorías con su vida útil y si son perecederas, y presentaciones. Cada producto tiene su unidad de conteo (kg, litros o unidades) y cada presentación guarda su peso en kg (UC-07).

• Terceros (soporte): personas naturales y jurídicas con sus roles de donante, proveedor y destinatario, vinculación de organizaciones y autorizaciones de tratamiento de datos (UC-08). Concentra los datos personales del sistema (CON-4).

• Análisis de vencimiento (núcleo del dominio; capa de soporte a la decisión): cálculo del riesgo por producto, alertas y parámetros de riesgo (UC-04, UC-10).

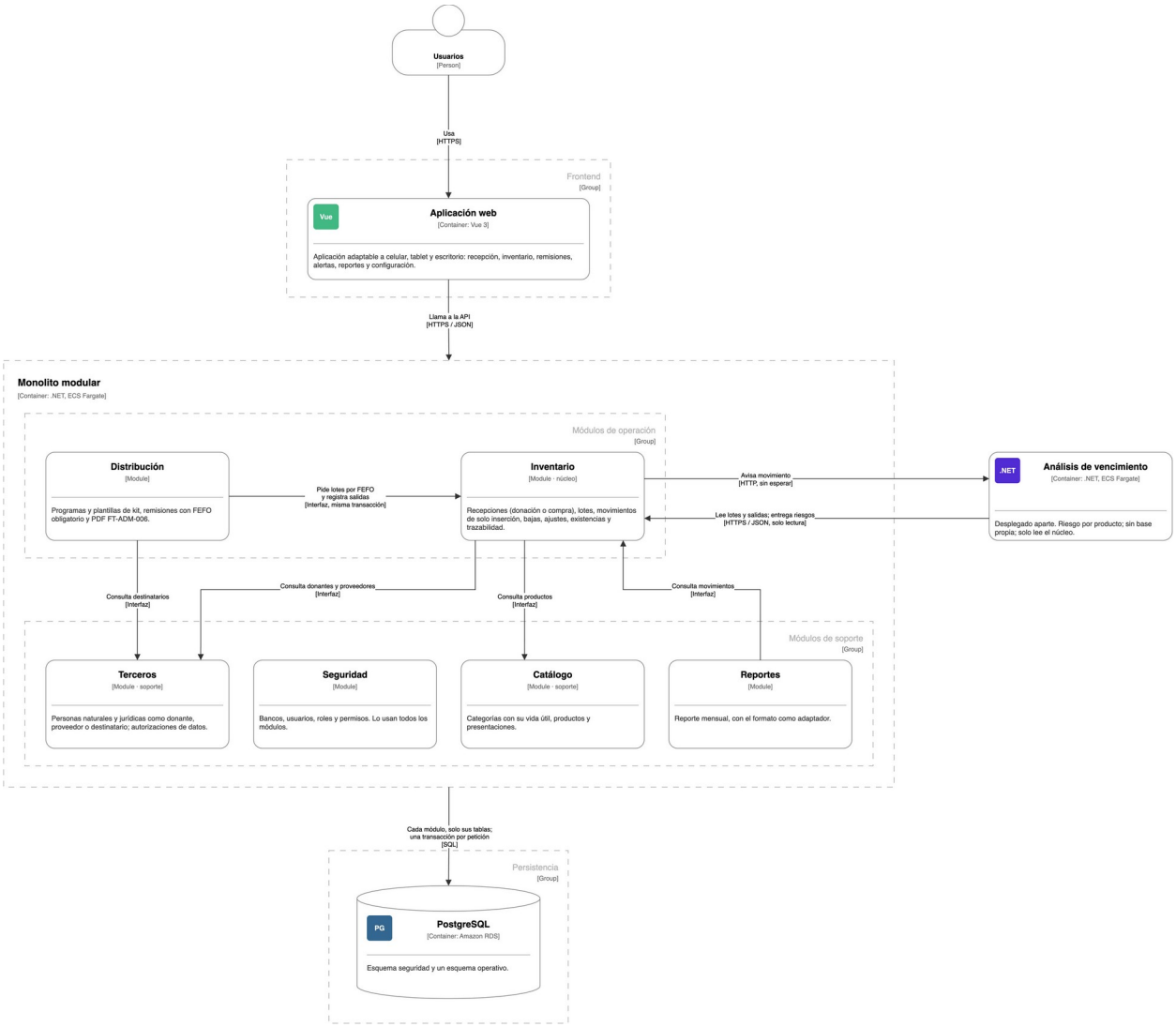
– Sin base propia: el contenedor de análisis calcula los riesgos y los devuelve al monolito.

– Dónde quedan las alertas y los parámetros: el monolito los guarda en el contexto delimitado Análisis de vencimiento del modelo de datos, como en la versión del 2026-10-12. Son tablas propias de ese contexto, separadas de las del núcleo operativo.

– Lo que nunca hace: el análisis no escribe en lotes, movimientos ni remisiones (CON-8).

• Reportes: reporte mensual en kg, convertido con el peso de cada presentación (UC-06).

• Seguridad: bancos, usuarios, roles e invitaciones; resuelve el banco activo de cada petición (UC-11, UC-12).



3.2 Patrones

3.2.1 Clean Architecture

Cada módulo se organiza en capas de dominio, aplicación, infraestructura y presentación, y las dependencias apuntan hacia el dominio. La persistencia, los formatos de archivo y el proveedor de identidad quedan en la infraestructura, detrás de interfaces. El formato del reporte mensual es un adaptador de salida del módulo de Reportes, así que un cambio de formato no toca el dominio.

3.2.2 Domain-Driven Design

Los módulos corresponden a contextos delimitados que salen del Event Storming. Las reglas críticas viven en agregados del dominio:

- el lote con sus movimientos;
- la remisión con sus renglones y su número de kits;
- la asignación FEFO, que la hace Inventario sobre sus lotes;

• la persona con sus roles.

3.2.3 CQRS

Se separan los comandos, que registran movimientos, de las consultas de existencias y trazabilidad. Las consultas se pueden optimizar con vistas de lectura sin tocar el modelo de escritura.

3.2.4 Unit of Work

Cada comando confirma en una sola transacción todos sus cambios, de modo que no queden registros a medias. Por ejemplo, una remisión con los movimientos de salida de sus lotes.

• Una transacción por petición: el comando que inicia la operación abre una transacción de PostgreSQL, y el contenedor de dependencias entrega esa misma conexión, como servicio scoped, a todos los módulos que participan.

• Contextos unidos: cada módulo usa su propio DbContext, unido a la misma transacción.

• Un solo commit: solo el comando que la abrió confirma o revierte.

• Dónde vive: la transacción compartida y la publicación de eventos internos tras el commit están en un proyecto técnico común (Ssd.Shared) que todos los módulos referencian y que no referencia a ninguno. No contiene reglas del negocio.

3.2.5 Comunicación entre módulos por interfaces

• Contrato publicado: cada módulo publica un contrato, es decir sus interfaces y DTO, en un proyecto aparte; lo demás del módulo es interno.

• Mismo proceso: dentro del monolito, los demás módulos usan ese contrato con llamadas en el mismo proceso y comparten la transacción de la petición.

• Eventos internos: las reacciones que no necesitan ir en la misma transacción se publican como eventos internos después del commit. Por ejemplo, "movimiento confirmado" dispara el aviso al análisis.

• Pruebas de arquitectura: fallan si un módulo usa clases internas de otro o sus tablas.

• El análisis: la capa de soporte a la decisión está en otro contenedor. Se comunica por HTTP con el contrato de Inventario, solo para leer, y nunca escribe en el núcleo.

3.2.6 Análisis disparado por movimiento con recálculo periódico

Cada vez que se confirma un movimiento de inventario, el monolito avisa al módulo de análisis por HTTP sin esperar su respuesta, y el análisis recalcula el riesgo de ese lote. Además, el análisis recalcula todos los lotes vigentes una vez al día, porque el riesgo cambia con el paso del tiempo aunque no haya movimientos, y también cada vez que arranca. Si el análisis está caído, la recepción no se bloquea y lo que no se calculó se recupera en el recálculo al arrancar, sin necesidad de una outbox. El aviso se envía después de confirmar la transacción, desde un proceso en segundo plano del monolito y con timeout corto, de modo que la petición de recepción nunca espera al análisis. Un cambio de vida útil de una categoría también envía un aviso para recalcular los lotes de esa categoría, y un cambio de umbral recalcula todo. Para el ritmo de salida, el análisis lee por el contrato de Inventario las existencias por lote y las salidas de la ventana configurada. Cada salida trae la marca de programa extraordinario, para poder excluirla.

3.2.7 Control de acceso basado en roles

Cada rol agrupa permisos definidos por el sistema (por ejemplo, registrar recepción o emitir remisión). Usuarios, roles y permisos viven en el esquema seguridad.

• Usuarios del banco: cada uno pertenece a un solo banco y tiene uno o varios roles de ese banco. El coordinador da de alta a los demás usuarios de su banco (UC-12).

• Administrador: rol de la plataforma, sin banco, con todos los permisos. Da de alta los bancos y a sus coordinadores (UC-11) y puede operar en cualquier banco.

• Banco activo: el backend decide en cada petición sobre qué banco trabaja; el front nunca lo impone.

– Usuario del banco: el banco sale de seguridad, buscado con el sub del token; cualquier banco que envíe el front se ignora.

– Administrador: el front envía el banco elegido en un encabezado (X-Banco-Id); el backend comprueba el rol y que el banco exista, y registra cada acción con ese banco.

– Se descartaron guardar el banco activo del administrador en seguridad (impide trabajar en dos bancos a la vez) y emitir un token con el banco mediante una Lambda de Cognito (más piezas y el cambio de rol tarda hasta que vence el token).

– En el prototipo solo existe Neiva: el administrador entra a Neiva sin selector, y el selector se construye junto con la resolución del esquema por banco que queda pospuesta.

• Datos personales: los del módulo de Terceros solo son visibles con el permiso correspondiente, asignado al coordinador y a la asistente administrativa. Esos datos son los de representantes, personas naturales que reciben ayuda, donantes y proveedores personas naturales, y la población atendida.

3.2.8 Alta de usuarios por invitación

Nadie entra a un banco sin invitación. Quien da de alta (el administrador para un coordinador, el coordinador para los demás) registra en seguridad el correo, el banco y el rol, y el monolito envía el correo de invitación con Amazon SES, con el enlace de la aplicación y la fecha de vencimiento.

• Por qué no lo envía Cognito: Cognito solo envía invitaciones a usuarios propios con contraseña. Un usuario que entra con Google no existe en Cognito hasta su primer inicio de sesión, así que el correo lo envía el sistema.

• Aceptar es entrar: con Google como proveedor de identidad no hay un paso aparte de aceptación. En el primer inicio de sesión, el backend no encuentra el sub en seguridad, busca una invitación pendiente y vigente para el correo verificado del token, vincula el sub y activa al usuario.

• Estados: Pendiente (al invitar), Activo (al vincular), Vencida (7 días sin entrar; se calcula con la fecha de vencimiento, sin tarea programada), Cancelada (anulada antes de entrar) e Inactivo (acceso retirado a un usuario activo). Una invitación vencida o cancelada se puede reenviar con nueva fecha y un nuevo correo.

• Sin invitación no hay acceso: cualquiera con cuenta de Google puede autenticarse en Cognito, pero sin invitación vigente el backend responde que no tiene acceso. No se usa una Lambda pre sign-up para rechazarlo antes; queda como mejora si se quiere evitar esos registros en Cognito.

3.2.9 Registro de autorizaciones de tratamiento de datos

Cada titular de datos personales tiene registrada su autorización: fecha, medio (formato firmado o aceptación digital), finalidades autorizadas, versión de la política de tratamiento y soporte escaneado en S3. Como regla del dominio de Terceros, no se guardan los datos personales de un titular sin una autorización registrada. Aplica también a la persona natural que recibe ayuda: firma la autorización en la misma entrega.

3.2.10 Minimización de datos personales

La población atendida se registra como cifras por organización y programa, no como personas identificadas, lo que basta para el reporte mensual. De una persona natural solo se piden tipo y número de documento y nombre. Sujeto a confirmación del banco.

3.3 Base de Datos

PostgreSQL en una sola base de datos, con un esquema seguridad (bancos, usuarios, roles e invitaciones) y un único esquema operativo para el banco de Neiva. Los movimientos de inventario son de solo inserción: los ajustes se registran como movimientos nuevos y la base bloquea UPDATE y DELETE sobre ellos. La base se cifra en reposo.

3.4 Servicios Cloud Seleccionados

Proveedor: AWS, región us-east-1.

Servicio	Uso
Amazon ECS con AWS Fargate	ejecuta el monolito .NET y el módulo de análisis como dos tareas separadas, sin administrar servidores.
NAT Gateway	salida a internet de las tareas en la subred privada (llaves JWKS de Cognito e imágenes de contenedores), sin exponerlas.
Application Load Balancer	entrada HTTPS al monolito (todo el tráfico va por HTTPS).
Amazon RDS for PostgreSQL	base de datos administrada, en una instancia pequeña y con respaldos automáticos y cifrado en reposo.
Amazon S3	cifrado en reposo; guarda remisiones en PDF, reportes exportados y adjuntos de la recepción (soporte del donante o factura), estos en un bucket privado con acceso solo por URL firmada.
Amazon S3 + Amazon CloudFront	publicación del frontend Vue 3 como sitio estático.
AWS Certificate Manager	emite y renueva sin costo el certificado TLS que usan CloudFront y el ALB para el HTTPS (CON-4). Se crea en us-east-1, la región que exige CloudFront.
Dominio	el banco tiene dominio propio, pero el equipo no tiene acceso ni permiso para usarlo. Por eso las pruebas usan un subdominio de atcmediator.com, un dominio del equipo. Los usuarios entran por ssd.atcmediator.com, que apunta a CloudFront, y el ALB se publica como origen.ssd.atcmediator.com para que CloudFront le hable por HTTPS. El certificado cubre ambos nombres y se valida con registros DNS en atcmediator.com. Cuando la aplicación entre en operación, se contratará un dominio propio para el proyecto.
Amazon Route 53	aloja la zona DNS de atcmediator.com, que ya existe en la cuenta del equipo. Tiene los registros alias de ssd.atcmediator.com hacia CloudFront y de origen.ssd.atcmediator.com hacia el ALB, y los registros con los que ACM valida el certificado. No agrega costo fijo, porque la zona ya se paga (estimación no verificada: las consultas a registros alias de recursos de AWS no se cobran).
Amazon SES	envío del correo de invitación, desde un remitente de ssd.atcmediator.com verificado con registros DKIM en Route 53. La cuenta debe salir del modo de pruebas (sandbox) de SES para enviar a cualquier destinatario. Costo despreciable para el volumen de invitaciones (estimación no verificada: alrededor de US$0,10 por cada 1.000 correos).
Amazon Cognito	autenticación de usuarios, con Google como proveedor de identidad. Solo identifica a la persona; las invitaciones, los roles, los permisos y el banco de cada usuario se gestionan en el esquema seguridad.

3.5 Stack Tecnológico

Lista de herramientas y tecnologías requeridas para la correcta implementación del sistema.

Categoría	Herramientas
Frontend	Vue 3
Backend	C# / .NET
Generación de PDF	QuestPDF
Base de datos	PostgreSQL
Nube	AWS
Pruebas de API	Postman
Desarrollo asistido por IA	OpenCode, Claude Code

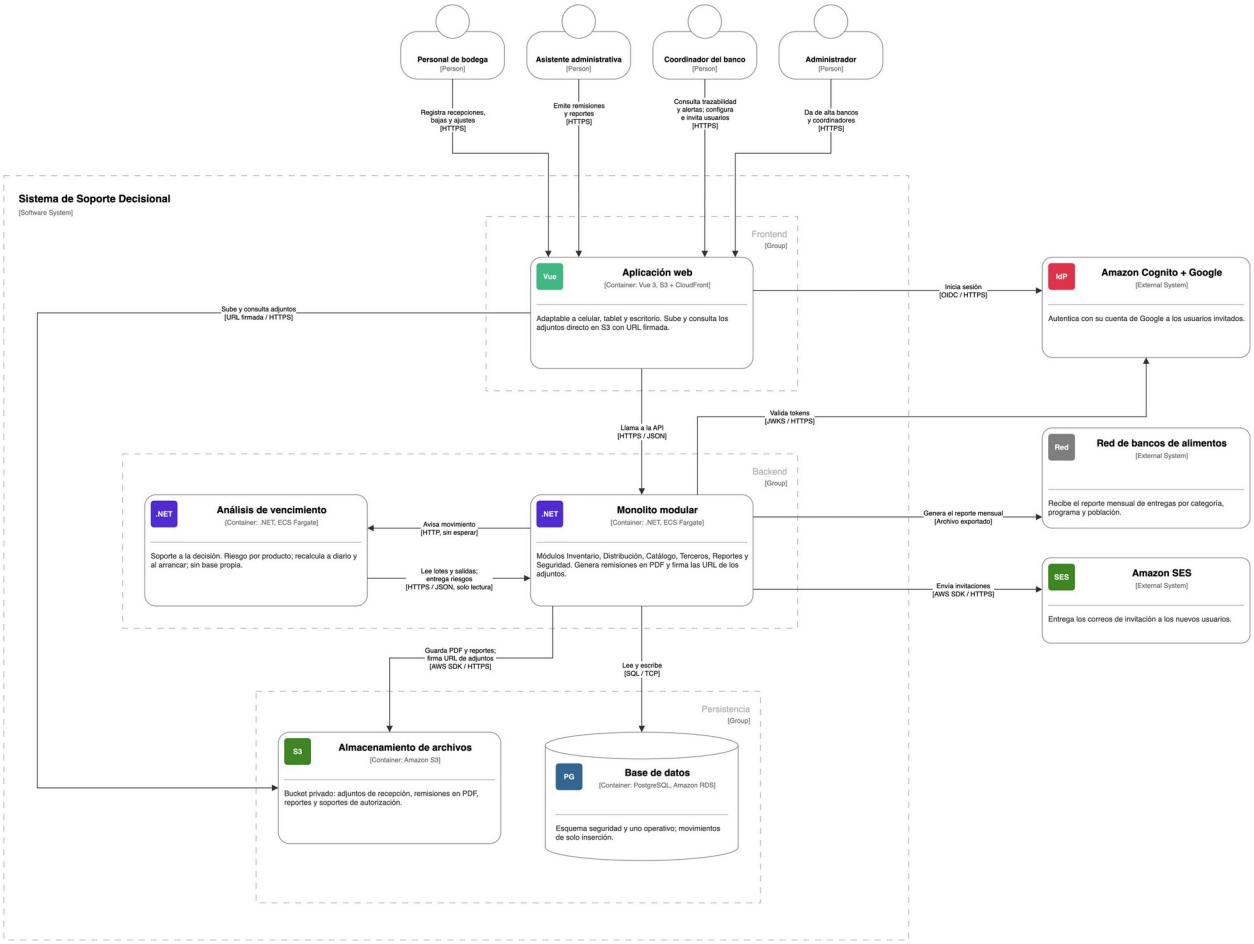
4. Definición de Arquitectura

4.1 Vista Funcional

4.1.1 Diagrama de contenedores

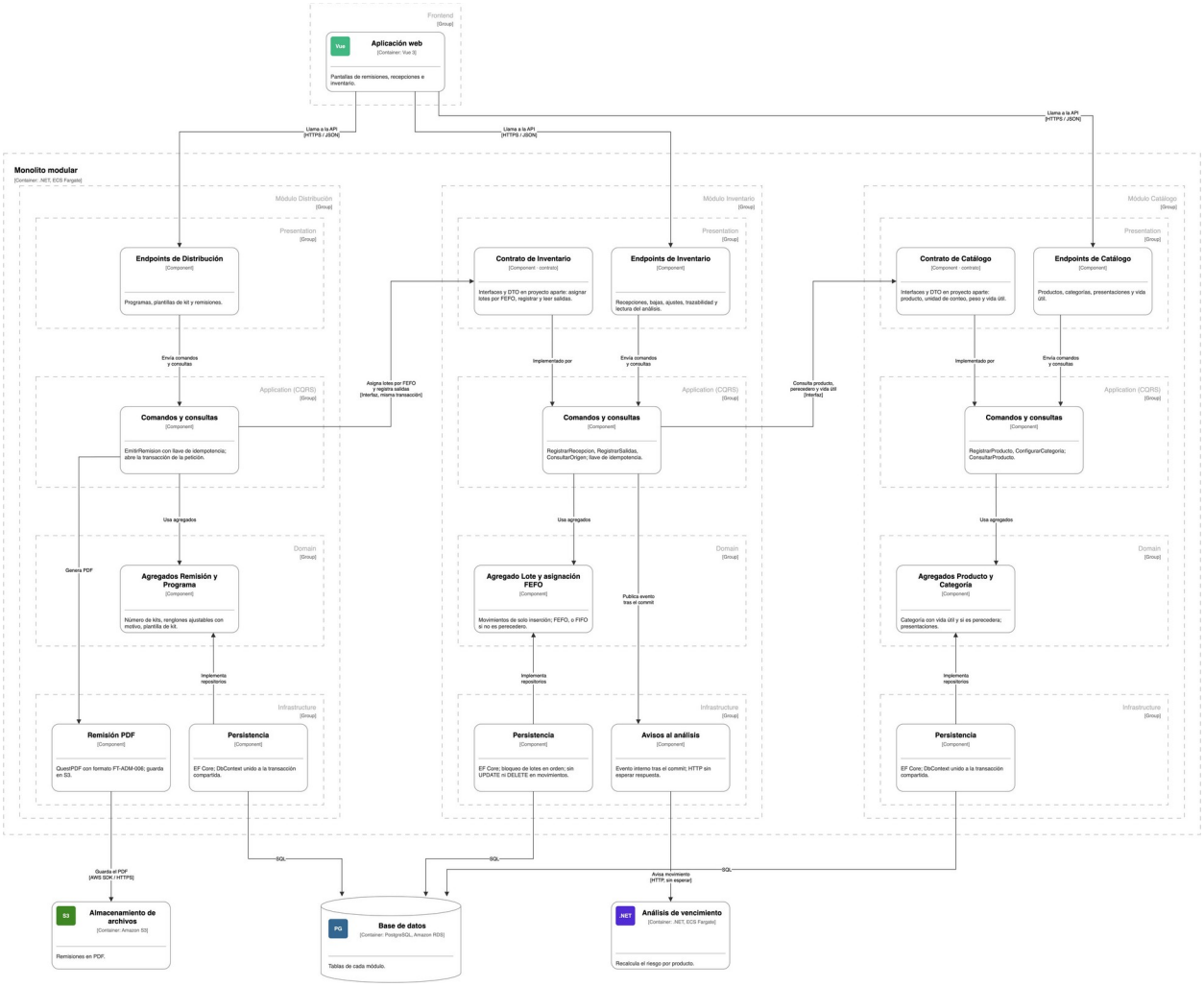
El sistema tiene cinco contenedores. La aplicación web en Vue 3, publicada en S3 y CloudFront y adaptable a celular, llama a la API del monolito modular y sube y consulta los adjuntos directamente en S3 con URL firmadas. El monolito modular (.NET en ECS Fargate) agrupa los módulos Inventario, Distribución, Catálogo, Terceros, Análisis de vencimiento, Reportes y Seguridad, genera las remisiones en PDF y envía las invitaciones por Amazon SES. El contenedor de análisis de vencimiento (.NET en ECS Fargate) recibe un aviso por cada movimiento sin que el monolito espere, lee las salidas por el contrato de Inventario y devuelve el riesgo por producto.

La base de datos PostgreSQL en Amazon RDS tiene el esquema seguridad y el esquema operativo del banco, con movimientos de solo inserción; un bucket privado de Amazon S3 guarda adjuntos de recepción, remisiones, reportes y soportes de autorización. Cognito con Google autentica a los usuarios invitados.



4.1.2 Diagrama de componentes

El diagrama muestra los módulos Distribución, Inventario y Catálogo del monolito, cada uno con sus capas Presentation, Application (comandos y consultas con CQRS), Domain e Infrastructure. Inventario y Catálogo publican su contrato en un proyecto aparte. Distribución emite la remisión y pide a Inventario, por su contrato y en la misma transacción, que asigne los lotes por FEFO y registre las salidas; Inventario consulta a Catálogo si el producto es perecedero y su vida útil. Tras el commit, Inventario avisa al análisis sin esperar respuesta.



4.2 Vista de Información (Base de Datos)

El modelo conceptual usa una base PostgreSQL con un esquema seguridad compartido (bancos, usuarios, roles, permisos, invitaciones y auditoría) y un esquema banco_<id> por banco con sus datos operativos; el prototipo crea solo el de Neiva. Dentro de cada esquema, un contexto por módulo: Catálogo (categoría, producto con su unidad de conteo y presentación con su peso), Inventario (recepción, lote y movimientos de solo inserción), Distribución (remisión, renglones, asignación de lotes por FEFO, programas y kits), Terceros (persona con roles de donante, proveedor o destinatario, organización, representante, población atendida y autorización de datos), Análisis de vencimiento (riesgo por producto, alertas y parámetros) y Reportes.

El modelo se recorrió con una semana de ejemplo que cubre los doce casos de uso: donación con fecha de empaque, estimada y sin fecha; riesgo de la leche; baja; entrega de emergencia a una persona; compra de arroz por arrobas; cuatro kits con FEFO repartido en dos lotes; compostaje; ajuste de un conteo; cambio de un parámetro de riesgo; invitación de un usuario; reporte mensual y trazabilidad de vuelta. No dejó huecos en el modelo; las reglas por confirmar con el banco quedaron como preguntas para el cliente.

Modelo de datos — Sistema de Soporte Decisional del Banco Diocesano de Alimentos de Neiva

(Conceptual Model Diagram)

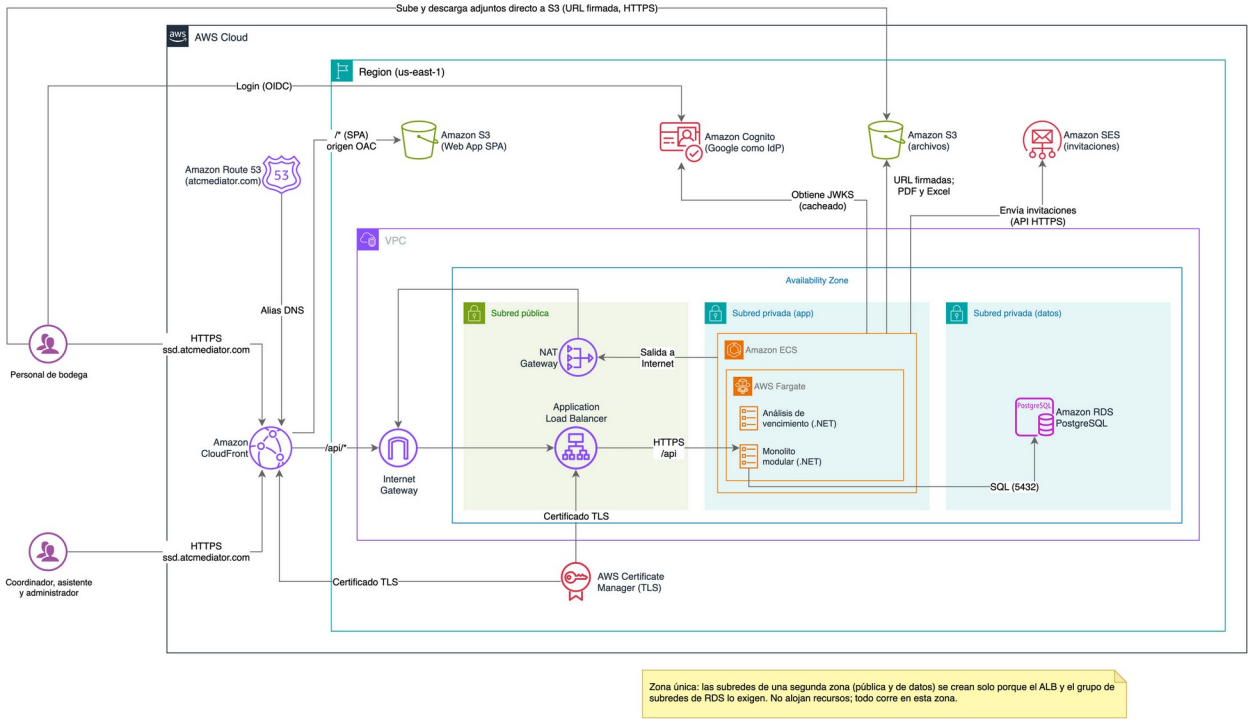
Descripción: Una base PostgreSQL con un esquema seguridad compartido y un esquema banco_<id> por banco con sus datos operativos; el prototipo crea solo el de Neiva (ADR-011). Dentro de cada esquema, un «Context» por módulo.

Vista: Información | Modelo: Datos Conceptual | Versión: 2.0 | Fecha: 2026-10-17



4.3 Vista de Despliegue

El sistema se despliega en AWS, región us-east-1, en una sola zona de disponibilidad. Los usuarios entran por ssd.atcmediator.com a CloudFront, que sirve la aplicación web desde S3 y envía /api/* por el Internet Gateway al Application Load Balancer de la subred pública. El monolito y el análisis corren como tareas de ECS Fargate en la subred privada de aplicación, con salida a internet por el NAT Gateway; RDS PostgreSQL está en la subred privada de datos. Los adjuntos suben y bajan directamente de S3 con URL firmadas. Cognito con Google, Amazon SES para las invitaciones, Certificate Manager y Route 53 completan el despliegue.



4.4 Vista de Desarrollo (Paquetes)

El repositorio del backend se organiza por módulo y por capa, con un proyecto de clases por capa de Clean Architecture y un proyecto de contratos por módulo. El frontend Vue 3 vive en su propio repositorio. Los nombres del código van en inglés; los diagramas y este documento usan los términos del negocio en español.

• Cada capa es un proyecto de clases. Dentro de un módulo: Presentation → Application → Domain; Infrastructure → Application y Domain. Domain no depende de nada.

• Entre módulos: un módulo solo referencia el proyecto Contracts de otro, nunca sus capas. Ejemplo: Distribution usa Ssd.Inventory.Contracts para que Inventory asigne lotes por FEFO.

• Todos los módulos pueden referenciar Ssd.Shared; este no referencia a ningún módulo ni contiene reglas del negocio.

• Ssd.Host referencia todos los módulos para registrarlos; ningún módulo referencia a Ssd.Host.

• El contenedor de análisis no referencia ningún módulo del monolito: lee a Inventory por HTTP con los DTO de Ssd.Inventory.Contracts.

• Ssd.ArchitectureTests hace cumplir estas reglas en cada compilación.

Glosario negocio → código: Seguridad: Banco · Usuario · Rol · Permiso · Invitación · Auditoría → FoodBank · User · Role · Permission · Invitation · AuditLog; Catálogo: Categoría · Producto · Presentación · Unidad de conteo · Vida

útil → Category · Product · Packaging · BaseUnit · ShelfLifeDays; Inventario: Recepción · Lote · Movimiento de inventario · Origen de la fecha · Baja · Ajuste · Compostaje → Receipt · Lot · InventoryMovement · ExpiryDateSource · WriteOff · Adjustment · Composting; Distribución: Remisión · Renglón de remisión · Asignación de lote · Programa · Componente del kit · Número de kits → DispatchNote · DispatchNoteLine · LotAllocation · Program · KitComponent · KitCount; Terceros: Persona · Rol de la persona (donante, proveedor, destinatario) · Organización · Representante · Población atendida · Autorización de datos → Party · PartyRole (Donor, Supplier, Recipient) · Organization · Representative · ServedPopulation · DataConsent; Análisis de vencimiento: Riesgo del producto · Alerta · Parámetro de riesgo · Ritmo de salida → ProductRisk · Alert · RiskParameter · OutflowRate; Reportes: Reporte mensual → MonthlyReport

```
ssd/
├── src/
│   ├── Ssd.Host/                          ← punto de entrada del monolito: arma los módulo…
│   ├── Ssd.Shared/                        ← solo piezas técnicas: transacción compartida, …
│   │
│   ├── Modules/
│   │   ├── Inventory/                     ← núcleo del dominio · Inventario
│   │   │   ├── Ssd.Inventory.Contracts/       ← asignar lotes por FEFO, registrar y leer sa…
│   │   │   ├── Ssd.Inventory.Domain/          ← agregado Lot, InventoryMovement, asignación…
│   │   │   ├── Ssd.Inventory.Application/     ← comandos y consultas: RegisterReceipt, Regi…
│   │   │   ├── Ssd.Inventory.Infrastructure/  ← EF Core (DbContext unido a la transacción),…
│   │   │   └── Ssd.Inventory.Presentation/    ← endpoints HTTP del módulo
│   │   ├── Distribution/                  ← Distribución
│   │   │   ├── Ssd.Distribution.Contracts/    ← interfaces y DTO públicos del módulo
│   │   │   ├── Ssd.Distribution.Domain/       ← DispatchNote, Program, KitComponent
│   │   │   ├── Ssd.Distribution.Application/  ← comandos y consultas: IssueDispatchNote…
│   │   │   ├── Ssd.Distribution.Infrastructure/ ← EF Core, PDF con QuestPDF, S3
│   │   │   └── Ssd.Distribution.Presentation/ ← endpoints HTTP del módulo
│   │   ├── Catalog/                       ← Catálogo
│   │   │   ├── Ssd.Catalog.Contracts/         ← producto, unidad de conteo, presentación co…
│   │   │   ├── Ssd.Catalog.Domain/            ← Product, Category, Packaging
│   │   │   ├── Ssd.Catalog.Application/       ← comandos y consultas: RegisterProduct, Conf…
│   │   │   ├── Ssd.Catalog.Infrastructure/    ← EF Core
│   │   │   └── Ssd.Catalog.Presentation/      ← endpoints HTTP del módulo
│   │   ├── Parties/                       ← Terceros
│   │   │   ├── Ssd.Parties.Contracts/         ← interfaces y DTO públicos del módulo
│   │   │   ├── Ssd.Parties.Domain/            ← Party con roles, Organization, DataConsent
│   │   │   ├── Ssd.Parties.Application/       ← comandos y consultas: RegisterRecipient…
│   │   │   ├── Ssd.Parties.Infrastructure/    ← EF Core
│   │   │   └── Ssd.Parties.Presentation/      ← endpoints HTTP del módulo
│   │   ├── ExpiryRisk/                    ← Análisis de vencimiento
│   │   │   ├── Ssd.ExpiryRisk.Contracts/      ← interfaces y DTO públicos del módulo
│   │   │   ├── Ssd.ExpiryRisk.Domain/         ← ProductRisk, Alert, RiskParameter
│   │   │   ├── Ssd.ExpiryRisk.Application/    ← comandos y consultas: ConfigureRiskParamete…
│   │   │   ├── Ssd.ExpiryRisk.Infrastructure/ ← EF Core, cliente HTTP del contenedor de aná…
│   │   │   └── Ssd.ExpiryRisk.Presentation/   ← endpoints HTTP del módulo
│   │   ├── Reporting/                     ← Reportes
│   │   │   ├── Ssd.Reporting.Contracts/       ← interfaces y DTO públicos del módulo
│   │   │   ├── Ssd.Reporting.Domain/          ← MonthlyReport
│   │   │   ├── Ssd.Reporting.Application/     ← comandos y consultas: GenerateMonthlyReport…
│   │   │   ├── Ssd.Reporting.Infrastructure/  ← EF Core, formato del reporte, S3
│   │   │   └── Ssd.Reporting.Presentation/    ← endpoints HTTP del módulo
│   │   └── Security/                      ← Seguridad
│   │       ├── Ssd.Security.Contracts/        ← usuario actual y banco activo
│   │       ├── Ssd.Security.Domain/           ← FoodBank, User, Role, Invitation
│   │       ├── Ssd.Security.Application/      ← comandos y consultas: InviteUser, ResolveAc…
│   │       ├── Ssd.Security.Infrastructure/   ← EF Core, Amazon SES
│   │       └── Ssd.Security.Presentation/     ← endpoints HTTP del módulo
│   │
│   └── Analysis/                          ← contenedor aparte (ECS Fargate): análisis de v…
│       ├── Ssd.Analysis.Domain/               ← reglas de riesgo
│       ├── Ssd.Analysis.Application/
```

```
│   ├── Ssd.Analysis.Infrastructure/      ← cliente HTTP del contrato de Inventory
│   └── Ssd.Analysis.Presentation/
│
└── tests/
    ├── Ssd.ArchitectureTests/            ← ningún módulo usa clases internas ni tablas de…
    ├── Ssd.<Module>.Tests/               ← pruebas de dominio y aplicación, una por módulo
    └── Ssd.IntegrationTests/             ← PostgreSQL real: transacción compartida, FEFO …
```

5. ADRs

Se documentan aquí únicamente las decisiones que implicaron un trade-off real entre alternativas viables. Las decisiones impuestas por el contexto o por restricciones están en la sección 3.

ID	ADR-001: Estilo arquitectónico
Contexto	Un equipo de tres personas con el plazo del trabajo de grado (CON-2), costo bajo (CON-3) y carga baja (AS-1); al mismo tiempo, una falla del análisis no debe bloquear la operación (QA-04), las reglas de decisión deben poder reemplazarse (QA-07) y la capa de decisión debe quedar separada del núcleo (CON-8).
Alternativas	**1. Microservicios** Un servicio desplegable por cada contexto del dominio, cada uno con su base de datos. **Ventajas** • Aislamiento de fallos y despliegue independiente por servicio. **Desventajas** • Mucha infraestructura y coordinación para tres personas; más costo de operación; consultas de trazabilidad (QA-01) repartidas entre servicios. **2. Monolito en capas** Una aplicación con capas técnicas (presentación, negocio, datos), sin límites por dominio. **Ventajas** • Lo más simple de construir y desplegar. **Desventajas** • El análisis queda acoplado al inventario (incumple QA-07 y CON-8) y una falla suya afecta la recepción (QA-04). **3. Monolito modular con el análisis en un contenedor aparte (elegida)** Una aplicación .NET con módulos por contexto del dominio, comunicados por interfaces; el análisis se despliega en un contenedor separado. La comunicación dentro del monolito se precisó en ADR-017 (2026-10-16): en el mismo proceso, no por HTTP. **Ventajas** • Un solo despliegue principal; límites claros entre módulos; la falla del análisis queda aislada y su modelo se reemplaza sin tocar el resto. **Desventajas** • Dos despliegues en vez de uno; la disciplina de no cruzar módulos depende del equipo.
Decisión	Monolito modular, con el análisis de vencimiento en un contenedor aparte.

ID	ADR-001: Estilo arquitectónico
Consecuencias	• Los contratos entre módulos deben definirse y respetarse desde el inicio. Si un módulo se extrae después, su contrato ya existe y se le pone HTTP detrás (ver ADR-017).

ID	ADR-003: Comunicación con el módulo de análisis
Contexto	El riesgo de vencimiento debe actualizarse con los movimientos de inventario y con el paso del tiempo (UC-04), pero una falla del análisis no puede bloquear la recepción y lo pendiente debe calcularse al recuperarse (QA-04); el análisis está en otro contenedor y se comunica con el monolito por HTTP.
Alternativas	**1. Llamada HTTP síncrona en cada movimiento** El inventario llama al análisis y espera su respuesta antes de confirmar. **Ventajas** • El riesgo queda calculado al instante. **Desventajas** • Si el análisis falla, la recepción se bloquea; incumple QA-04. **2. Outbox con entrega HTTP y reintentos** El inventario guarda el aviso en una tabla dentro de su transacción y un proceso en segundo plano lo entrega con reintentos. **Ventajas** • Ningún aviso se pierde. **Desventajas** • Una tabla, un proceso y una lógica de reintentos más que construir; no cubre el paso del tiempo por sí sola. **3. Mensajería con Amazon SQS** El inventario publica los cambios en una cola que consume el análisis. **Ventajas** • Entrega garantizada y desacople total. **Desventajas** • Otro servicio que operar y pagar; rompe la convención de comunicación por HTTP. **4. Aviso HTTP sin espera más recálculo diario y al arrancar (elegida)** El monolito avisa al análisis sin esperar respuesta en cada movimiento; el análisis recalcula todos los lotes vigentes una vez al día y cada vez que arranca. **Ventajas** • La recepción nunca espera al análisis; lo perdido se recupera solo; cubre el paso del tiempo; sin infraestructura adicional. **Desventajas** • Si el análisis está caído, sus alertas se atrasan hasta que se recupere.
Decisión	Aviso HTTP sin espera por movimiento, con recálculo completo diario y al arrancar.

ID	ADR-003: Comunicación con el módulo de análisis
Consecuencias	• El recálculo completo debe tolerar el volumen de lotes vigentes (AS-1). • Desde 2026-10-16, el contrato de Inventario debe exponer las salidas por producto en una ventana de días, con la marca de programa extraordinario, para calcular el ritmo de salida.

ID	ADR-004: Registro del inventario
Contexto	Cada kg entregado debe vincularse a su entrada (QA-01) y las correcciones no pueden borrar historia (QA-02); se usa CQRS para separar escrituras y consultas.
Alternativas	**1. Saldos actualizables con tabla de auditoría** Cada lote guarda su cantidad actual y los cambios se copian a una bitácora. **Ventajas** • Consultas de existencias directas. **Desventajas** • La historia es secundaria y puede divergir del saldo; la trazabilidad depende de la bitácora. **2. Event Sourcing** El estado de cada lote se reconstruye a partir de su secuencia de eventos. **Ventajas** • Historia completa por diseño. **Desventajas** • Curva de aprendizaje y complejidad altas para el plazo (CON-2); requiere proyecciones para todas las consultas. **3. Movimientos inmutables de solo inserción (elegida)** Cada entrada, salida, baja o ajuste es un movimiento nuevo sobre un lote; nunca se actualiza ni se borra. Las existencias se obtienen de los movimientos, con vistas de lectura (CQRS) si hace falta rendimiento. **Ventajas** • Trazabilidad e historia sin piezas adicionales; modelo relacional conocido por el equipo. **Desventajas** • Las existencias requieren sumar movimientos o mantener vistas de lectura.
Decisión	Movimientos inmutables de solo inserción.
Consecuencias	• La base no debe permitir UPDATE ni DELETE sobre la tabla de movimientos (por permisos o por triggers). QA-01 (menos de 5 s) se verifica con vistas de lectura si la suma directa no alcanza.

ID	ADR-005: Reglas de negocio configurables
Contexto	La composición de los kits y la vida útil por categoría cambian con frecuencia y deben aplicarse sin tocar código ni desplegar (QA-06); el riesgo se calcula con reglas sobre días al vencimiento, existencias y ritmo de salida (CON-6).

ID	ADR-005: Reglas de negocio configurables
Alternativas	**1. Reglas en el código** Kits y vida útil definidos como constantes o configuración desplegada. **Ventajas** • Lo más simple. **Desventajas** • Cada cambio exige desarrollar y desplegar; incumple QA-06. **2. Motor de reglas externo** Un motor de reglas genérico evalúa reglas escritas por el usuario. **Ventajas** • Máxima flexibilidad. **Desventajas** • Sobredimensionado para dos tipos de regla; más complejidad y aprendizaje. **3. Parámetros editables desde la aplicación (elegida)** La vida útil es un atributo de la categoría y la composición de kits es un dato por programa; el coordinador edita la vida útil desde el módulo de catálogo y la composición de kits desde el módulo de distribución. **Ventajas** • Cambio inmediato sin despliegue; simple de construir. **Desventajas** • Solo cubre las reglas previstas; una regla nueva requiere desarrollo.
Decisión	Reglas como parámetros: vida útil en el catálogo y composición de kits en distribución.
Consecuencias	• Los cambios de parámetros deben quedar registrados (quién y cuándo), para no perder la trazabilidad de por qué un kit tuvo cierta composición. • Desde 2026-10-16 también son parámetros los umbrales de riesgo y la ventana del ritmo de salida, que se editan en el módulo de análisis (UC-10). • Un cambio de plantilla solo afecta a las remisiones futuras, porque cada remisión guarda sus propios renglones.

ID	ADR-007: Plataforma de despliegue en AWS
Contexto	Dos contenedores (monolito y análisis) con fallos aislados (QA-04), recuperación sin intervención (QA-05), costo bajo (CON-3) y un equipo pequeño (CON-2). En la versión aprobada el 2026-10-12 también pesaban las conexiones SSE largas, que se retiraron con ADR-008.
Alternativas	**1. Una instancia Lightsail o EC2 con Docker Compose** Los contenedores, y opcionalmente PostgreSQL, en un solo servidor. **Ventajas** • El menor costo. **Desventajas** • El servidor se administra a mano; si cae, cae todo; respaldos manuales.

ID	ADR-007: Plataforma de despliegue en AWS
	2. AWS App Runner Contenedores administrados con escalado automático. **Ventajas** • Muy poca configuración. **Desventajas** • Corta las peticiones a los 120 s, lo que rompía el SSE. Revisado el 2026-10-16, ya sin SSE: sigue necesitando un conector de VPC y NAT para llegar a RDS y a Cognito; el análisis quedaría como servicio con dirección pública; ahorra solo el ALB. **3. Amazon ECS con Fargate, Application Load Balancer y RDS for PostgreSQL (elegida)** Una tarea Fargate por contenedor, entrada por ALB y base administrada. **Ventajas** • Sin servidores que administrar; tareas aisladas y privadas; reinicio automático de tareas; respaldos automáticos. **Desventajas** • Costo fijo mayor por el ALB y el RDS.
Decisión	ECS Fargate con ALB y RDS for PostgreSQL.
Consecuencias	• Se usa una sola tarea del monolito, y el servicio de ECS la reinicia sola si falla (QA-05). • Revisado el 2026-10-16: se mantiene la decisión. • Costo estimado de unos US$ 80–100 al mes: ALB ~US$ 16–20, NAT ~US$ 32 más tráfico, dos tareas pequeñas ~US$ 9 cada una, más RDS. Es una estimación no verificada con la calculadora de AWS. El prototipo se despliega en una sola zona de disponibilidad; el ALB y el grupo de subredes de RDS exigen subredes en dos zonas, que se crean sin duplicar las tareas. El NAT Gateway agrega un costo fijo mensual (CON-3).

ID	ADR-010: Aplicación de FEFO en la remisión
Contexto	Cada renglón de una remisión debe salir del lote que vence primero para reducir el desperdicio (UC-03), y cada kg entregado debe quedar vinculado al lote del que realmente salió (QA-01). El arquitecto decidió FEFO obligatorio el 2026-10-16. Reemplaza a ADR-009.
Alternativas	**1. FEFO sugerido con motivo obligatorio (ADR-009)** El sistema preselecciona el lote; la asistente puede elegir otro indicando un motivo. **Ventajas** • Atiende pedidos o excepciones en el momento del despacho. **Desventajas** • Deja de ser FEFO en la práctica; las excepciones dependen del criterio de quien despacha. **2. FEFO obligatorio con excepciones vía baja o ajuste (elegida)** El sistema asigna el lote que vence primero (FIFO para no perecederos, fecha

ID	ADR-010: Aplicación de FEFO en la remisión
	estimada para perecederos sin fecha) y nadie lo puede cambiar. Si el lote está dañado o no aparece, se registra una baja o un ajuste en Inventario (UC-05) y FEFO pasa solo al siguiente. **Ventajas** • Nunca sale un lote nuevo antes que uno viejo; el registro coincide con lo que salió; las bajas y ajustes miden las pérdidas en bodega. **Desventajas** • No atiende el pedido de un lote específico (AS-7).
Decisión	FEFO obligatorio, ejecutado por Inventario (dueño de los lotes) cuando Distribución le pide registrar las salidas de los renglones finales de la remisión, en la misma transacción (ADR-017). Las excepciones se resuelven en Inventario mediante baja o ajuste.
Consecuencias	• La asignación se hace al confirmar la remisión, sobre los renglones finales: los de la plantilla multiplicada por el número de kits, más los ajustes. • Un renglón puede repartirse en varios lotes. • Desaparecen la lista de motivos de excepción y su consulta. • La asignación se protege frente a remisiones simultáneas sobre el mismo lote con ADR-013.

ID	ADR-011: Preparación para varios bancos
Contexto	Desde la versión del 2026-10-16 de drivers.md, operar con otros bancos es una proyección (AS-5) sin escenario que validar, pero el diseño no debe cerrar esa puerta. El equipo y el plazo son limitados (CON-2) y el costo debe ser bajo (CON-3). Reemplaza a ADR-002.
Alternativas	**1. Mantener ADR-002 completo** Esquema seguridad y un esquema por banco, elegido en cada transacción según el usuario, con migraciones que recorren todos los esquemas. **Ventajas** • Un segundo banco se habilita por configuración. **Desventajas** • Resolvedor, migraciones múltiples y pruebas de aislamiento que el prototipo, con un solo banco, no usa; riesgo de exponer datos de otro banco por una conexión reutilizada. **2. Estructura separada con resolución dinámica pospuesta (elegida)** Esquema seguridad con la tabla de bancos y un único esquema operativo para Neiva. No se construyen el resolvedor por usuario ni las migraciones múltiples. **Ventajas** • Casi sin costo hoy; habilitar otro banco se concentra en un punto de la infraestructura (el acceso a datos), sin tocar el dominio. **Desventajas** • Un segundo banco requiere desarrollo y una migración, no solo configuración. **3. Un solo esquema sin separación**

ID	ADR-011: Preparación para varios bancos
	Todas las tablas juntas, sin esquema seguridad. **Ventajas** • Lo más simple. **Desventajas** • Cierra la puerta a otros bancos; mezcla seguridad y datos operativos.
Decisión	Esquema seguridad y un único esquema operativo; la resolución del esquema por banco queda pospuesta.
Consecuencias	• El acceso a datos de cada módulo toma el nombre del esquema operativo de la configuración, en un solo punto, para que agregar el resolvedor después no toque el dominio. • Cada usuario sigue perteneciendo a un banco en seguridad. • El riesgo R-08 de la validación se retira.

ID	ADR-012: Validación de la integridad de los datos
Contexto	Todo dato incompleto o inconsistente debe rechazarse en la captura, indicando qué corregir y sin guardar nada (QA-03). Los casos son lote perecedero sin fecha en una categoría sin vida útil, salida mayor que las existencias, producto fuera del catálogo, vencido como aprovechado y remisión sin destinatario. El registro lo hace personal con poca formación técnica (CON-7).
Alternativas	**1. Solo en el dominio** Los agregados validan las reglas; la base no tiene restricciones. **Ventajas** • Mensajes claros; las reglas viven en un solo lugar. **Desventajas** • Un error de código o un acceso directo a la base puede guardar datos inconsistentes. **2. En el dominio y en la base de datos (elegida)** El dominio valida y entrega el mensaje de qué corregir. La base agrega una segunda barrera con obligatorios, llaves foráneas, CHECK (perecedero exige fecha de vencimiento, cantidades mayores que cero) y únicos. **Ventajas** • Mensajes claros y garantía de 0 registros inconsistentes aunque falle el código. **Desventajas** • Algunas reglas se declaran dos veces y deben mantenerse alineadas. **3. Solo en la base de datos** Restricciones y triggers; el dominio queda delgado. **Ventajas** • Garantía fuerte. **Desventajas** • Los errores de la base no indican qué corregir; las reglas quedan repartidas en SQL.

ID	ADR-012: Validación de la integridad de los datos
Decisión	Validación en el dominio con una segunda barrera de restricciones en la base de datos.
Consecuencias	• Las pruebas de QA-03 intentan cada caso por la API y directamente contra la base. • La validación en el formulario es una ayuda de usabilidad, no una barrera. • La regla "salida mayor que las existencias" no tiene una columna con CHECK, porque las existencias salen de los movimientos (ADR-004). La resuelve ADR-013.

ID	ADR-013: Salidas simultáneas sobre el mismo lote
Contexto	Dos operaciones simultáneas sobre el mismo lote (dos remisiones, o una remisión y una baja) no pueden dejar existencias negativas (QA-03). Las existencias se calculan a partir de movimientos de solo inserción (ADR-004), y la carga es baja (AS-1).
Alternativas	**1. Bloqueo de los lotes al confirmar (elegida)** La transacción de confirmación bloquea las filas de los lotes del producto (SELECT … FOR UPDATE), recalcula existencias, asigna por FEFO e inserta los movimientos; la operación concurrente espera y ve el saldo descontado. **Ventajas** • Simple y exacto; mantiene los movimientos como única fuente de verdad. **Desventajas** • La operación concurrente espera; insignificante con la carga de AS-1. **2. Aislamiento serializable** PostgreSQL detecta el conflicto y rechaza una de las transacciones, que se reintenta. **Ventajas** • Sin bloqueos explícitos. **Desventajas** • Reintentos en cada comando; si fallan, el error llega al usuario. **3. Saldo guardado por lote con CHECK** Cada lote guarda su saldo, actualizado en la misma transacción, y la base impide el negativo. **Ventajas** • Garantía en la base. **Desventajas** • Es la alternativa que ADR-004 descartó: un dato actualizable que puede apartarse de los movimientos.
Decisión	Bloqueo pesimista de los lotes del producto en la transacción que confirma una salida, una baja o un ajuste.

ID	ADR-013: Salidas simultáneas sobre el mismo lote
Consecuencias	• Todos los comandos que descuentan existencias bloquean los lotes en el mismo orden (por vencimiento y luego por identificador), para evitar interbloqueos. • QA-03 se prueba con dos confirmaciones simultáneas sobre un lote con existencias justas.

ID	ADR-014: Confirmaciones sin duplicar al reintentar
Contexto	Si el servidor falla mientras se confirma una recepción o una remisión, el usuario puede reintentar, y el sistema no puede dejar registros perdidos, a medias ni duplicados (QA-05). Unit of Work ya garantiza que cada operación queda completa o no queda.
Alternativas	**1. Llave de idempotencia generada en el formulario (elegida)** Al abrir el formulario de una recepción, remisión, baja o ajuste, el navegador genera un identificador único. La confirmación lo envía y la base lo guarda con restricción de único. Si llega repetido, el servidor devuelve el registro ya creado. **Ventajas** • Distingue con exactitud un reintento de una operación nueva; poco código, en un filtro común a los comandos. **Desventajas** • El formulario debe conservar la llave hasta recibir la confirmación. **2. Detección por datos parecidos** Se rechaza una operación igual a otra reciente. **Ventajas** • No cambia el formulario. **Desventajas** • Falsos positivos con operaciones reales repetidas; reglas difusas. **3. Sin protección** El usuario revisa las últimas operaciones antes de reintentar. **Ventajas** • Nada que construir. **Desventajas** • Depende de la persona; incumple QA-05.
Decisión	Llave de idempotencia generada en el formulario y guardada con restricción de único junto a la operación.
Consecuencias	• La llave se guarda en la misma transacción que la operación, así que un reintento después de una caída encuentra la operación completa o no encuentra nada. • QA-05 se prueba matando la tarea durante una confirmación y reintentando.

ID	ADR-015: Almacenamiento de los adjuntos de la recepción
Contexto	La recepción puede llevar un adjunto opcional, el documento del donante o la factura de la compra. Cuando existe, es evidencia de la entrada y forma parte de su traza (QA-01). Como todo el historial, es inmutable: nadie puede cambiarlo ni borrarlo (QA-02). Contiene datos de terceros (CON-4) y el costo debe ser bajo (CON-3).
Alternativas	**1. S3 privado con acceso solo por URL firmada (elegida)** El archivo se guarda en un bucket privado; el servidor genera una URL firmada de subida, de pocos minutos y con una llave única por adjunto, y el navegador sube directo a S3; para consultarlo, el servidor genera una URL firmada de lectura de pocos minutos, solo para usuarios con permiso; la subida es condicional (no sobrescribe una llave existente), y la aplicación no tiene permisos de borrar ni de sobrescribir. **Ventajas** • Barato; el archivo no pasa por el servidor ni infla la base; la inmutabilidad la impone la configuración de S3 y de permisos, no el código. **Desventajas** • El archivo y la recepción no comparten transacción; pueden quedar archivos huérfanos de recepciones no confirmadas. **2. Dentro de PostgreSQL** El archivo se guarda como binario en una tabla. **Ventajas** • Misma transacción que la recepción. **Desventajas** • La base y sus respaldos crecen con cada archivo; el almacenamiento de RDS es más caro que el de S3; la inmutabilidad depende de bloquear UPDATE y DELETE también en esa tabla. **3. S3 con Object Lock** AWS impide borrar o sobrescribir durante un periodo de retención, incluso a los administradores. **Ventajas** • Inmutabilidad garantizada por AWS también frente a la cuenta. **Desventajas** • Más configuración; en modo estricto nadie puede borrar un archivo subido por error; excesivo para el prototipo.
Decisión	S3 privado con acceso solo por URL firmada de corta duración, subida condicional y sin permisos de borrado ni sobrescritura para la aplicación.
Consecuencias	• La URL de subida apunta a una carpeta temporal, y al confirmar la recepción el archivo pasa a su ubicación definitiva. Una regla de ciclo de vida de S3 borra lo temporal después de un día. • La recepción guarda la llave del archivo. • Solo quien administra la cuenta de AWS podría alterar un adjunto. Es un riesgo aceptado para el prototipo, y si hace falta cerrarlo, la vía es Object Lock.

ID	ADR-016: Interfaz para el registro en bodega
Contexto	El personal de bodega, a veces voluntario y con poca formación técnica, registra durante la recepción desde celular o tablet, con los términos de los formatos FT-ADM (CON-7). Se supone conexión estable en bodega (AS-2). El equipo es de tres personas (CON-2).
Alternativas	**1. Web adaptable, pensada primero para celular (elegida)** El mismo frontend Vue 3 con pantallas adaptables. Las de bodega se diseñan para celular, con controles grandes, pocos campos, términos del FT-ADM y validación mientras se escribe. **Ventajas** • Un solo código; nada que instalar. **Desventajas** • Sin conexión no funciona. **2. Aplicación web instalable (PWA) con modo sin conexión** Lo mismo, instalable y con caché local para registrar sin conexión. **Ventajas** • Tolera la falta de internet. **Desventajas** • Registrar sin conexión y sincronizar después choca con FEFO obligatorio (ADR-010) y con el bloqueo de lotes (ADR-013); mucho trabajo para un supuesto no refutado. **3. App nativa o híbrida** Una aplicación aparte para bodega. **Ventajas** • Mejor integración con el dispositivo. **Desventajas** • Segundo código y segundo despliegue (CON-2).
Decisión	Web adaptable en Vue 3, con las pantallas de bodega diseñadas primero para celular.
Consecuencias	• Las pantallas de recepción, bajas y ajustes se prueban en un celular con personal del banco. • Si AS-2 resulta falso, se evalúa pasar a una PWA sin rehacer el frontend; queda como riesgo para la validación.

ID	ADR-017: Comunicación y transacciones entre módulos
Contexto	Confirmar una remisión escribe en dos módulos: Distribución guarda la remisión, e Inventario bloquea lotes (ADR-013), asigna por FEFO (ADR-010) e inserta los movimientos. Las dos cosas deben quedar juntas o no quedar (QA-05). Los módulos no deben conocer el interior de los demás (QA-07, CON-8). Hay un solo monolito y una sola base (ADR-001, ADR-011), la carga es baja (AS-1) y el equipo es pequeño (CON-2). En la versión del 2026-10-12, los módulos del monolito se comunicaban por HTTP.
Alternativas	**1. Interfaces en el mismo proceso con transacción compartida (elegida)** Cada módulo publica un contrato de interfaces. Los demás lo llaman dentro del

ID	ADR-017: Comunicación y transacciones entre módulos
	proceso y todos participan en una transacción de PostgreSQL por petición, que confirma solo el comando que la abrió. **Ventajas** • Consistencia inmediata, sin operaciones a medias por construcción; lo más simple de construir y probar. **Desventajas** • Acopla a los módulos en el momento de confirmar; extraer un módulo obliga a rediseñar sus operaciones compartidas. **2. Transacción propia por módulo, coordinada por eventos con outbox** Distribución guarda la remisión como pendiente y publica un evento. Inventario lo procesa en su transacción y responde, y la remisión se confirma o se rechaza. **Ventajas** • Módulos independientes, listos para bases o despliegues separados. **Desventajas** • Por un tiempo existe una remisión sin su salida, lo que va contra QA-05; outbox, procesador, estados y compensaciones que construir. **3. HTTP entre los módulos del monolito (versión del 2026-10-12)** Cada módulo consume los contratos de los demás por HTTP. **Ventajas** • El límite entre módulos lo impone la red. **Desventajas** • Cada llamada es una transacción aparte, con los mismos problemas que la alternativa 2, más latencia y serialización dentro de una sola aplicación. **4. Unir Distribución e Inventario en un módulo** La remisión y los movimientos viven en el mismo módulo. **Ventajas** • Transacción trivial. **Desventajas** • Mezcla "qué hay" con "a quién se entrega" y debilita los límites del dominio.
Decisión	Dentro del monolito, comunicación por interfaces en el mismo proceso, con una transacción por petición. HTTP solo entre el monolito y el análisis.
Consecuencias	• La transacción se mantiene corta: los lotes se bloquean solo al asignar FEFO, insertar y confirmar, nunca mientras se edita el formulario. • Los eventos internos se publican después del commit. Si se pierde un aviso al análisis, lo recupera el recálculo de ADR-003. • Las pruebas de arquitectura vigilan que ningún módulo use el interior de otro. • Si algún módulo se extrae como servicio, sus operaciones compartidas pasan a la alternativa 2.

6. Validación de la Arquitectura

La arquitectura se evaluó contra los escenarios de calidad con un análisis inspirado en ATAM: para cada escenario se revisaron el enfoque que lo atiende, los puntos de sensibilidad, los compromisos y la prueba con que se validará.

6.1 Análisis por Escenario

Escenario	Enfoque	Sensibilidad y compromisos	Prueba
QA-01	movimientos inmutables (ADR-004); FEFO obligatorio ejecutado por Inventario (ADR-010); cada renglón de la remisión se cubre con uno o varios lotes (AsignacionLote), cada lote apunta a su recepción y esta a su persona y su adjunto inmutable en S3 (ADR-015); consultas separadas del modelo de escritura (CQRS).	índices sobre IdLote, IdRenglon e IdRecepcion; la cadena ya no es recursiva porque no hay fraccionamiento. Compromiso: las existencias se calculan sumando movimientos; se gana historia completa a cambio de consultas de agregación, mitigables con vistas de lectura. No-riesgo: con la carga de AS-1 (~20 remisiones por día) la consulta recorre 4 tablas con pocas filas por entrega. Riesgos: R-01.	cargar un año de datos sintéticos al volumen de AS-1 y medir que la consulta de origen responda en menos de 5 s en el percentil 95 y sin eslabones faltantes. Caso base: el paso 12 de analysis/walkthrough-inventory.md.
QA-02	movimientos de solo inserción con UPDATE y DELETE bloqueados en la base (ADR-004); un ajuste es un movimiento nuevo con IdMovimientoCorregido.	los privilegios del usuario de base de datos con que se conecta la aplicación. Riesgos: R-02.	intentar UPDATE y DELETE sobre movimientos con el usuario de la aplicación (deben fallar) y registrar un ajuste (paso 8 del flujo): queda como movimiento nuevo y el original intacto.
QA-03	validación en los agregados del dominio con restricciones en la base como segunda barrera (ADR-012); bloqueo pesimista de los lotes del producto al confirmar una salida, baja o ajuste (ADR-013); una sola transacción por petición (Unit of Work, ADR-017).	el orden en que se bloquean los lotes (si dos transacciones los toman en distinto orden, se bloquean mutuamente); que todos los módulos usen la transacción compartida de Ssd.Shared. Compromiso: el bloqueo serializa las salidas del mismo producto; con la carga de AS-1 la espera es imperceptible. Riesgos: R-07, R-08, R-15.	dos remisiones simultáneas que piden más de lo que hay de un producto: una se confirma y la otra se rechaza indicando qué corregir; 0 existencias negativas. Intentar guardar un lote perecedero sin fecha y sin vida útil en su categoría: se rechaza.

Escenario	Enfoque	Sensibilidad y compromisos	Prueba
QA-04	análisis en una tarea Fargate aparte (ADR-001, ADR-007); evento interno tras el commit y aviso HTTP sin espera (ADR-003, ADR-017); recálculo completo diario y al arrancar.	que el aviso salga después del commit y con timeout corto; si se espera dentro de la petición, la falla del análisis vuelve a bloquear la recepción. Compromiso: con el análisis caído las alertas se atrasan hasta que se recupera; se acepta a cambio de no usar broker ni outbox. Riesgos: R-03, R-11.	detener la tarea de análisis, registrar 50 recepciones y 10 remisiones (0 fallan o se demoran), reiniciarla y verificar que recalcula el riesgo de los productos afectados.
QA-05	reinicio automático de la tarea por ECS (ADR-007); una transacción por petición entre módulos (ADR-017); llave de idempotencia generada en el formulario y guardada con restricción de único (ADR-014).	que el reintento del formulario reutilice la misma llave; que ningún efecto externo (PDF en S3, correo) ocurra antes del commit. Compromiso: el PDF de la remisión se genera después del commit; si falla, se regenera desde los datos, que son la fuente de verdad. Riesgos: R-08.	detener la tarea mientras se confirma una remisión y reintentar con la misma llave: queda una sola remisión, completa o ninguna, y sus movimientos completos.
QA-06	vida útil y unidad de conteo en Catálogo, composición de kits en Distribución, umbrales y ventana del ritmo en Análisis de vencimiento, todos como datos editables (ADR-005); un cambio dispara el recálculo; el valor anterior queda en Auditoría.	el alcance de un cambio de vida útil. Decidido para el MVP: aplica también a los lotes ya recibidos, porque la fecha estimada no se guarda; se calcula como fecha de recepción + vida útil vigente de la categoría. Riesgos: R-04.	cambiar la ventana del ritmo, la vida útil de una categoría y la composición de un kit desde la aplicación (paso 9 del flujo); verificar sin despliegue que las alertas y la siguiente remisión usan el valor nuevo en menos de 1 hora.
QA-07	el análisis está en su propio contenedor, sin base propia, detrás de un contrato HTTP; solo lee el contrato de Inventario (ADR-001, ADR-003, ADR-017, Clean Architecture).	la estabilidad del contrato: salidas por producto en una ventana de días, con la marca de programa extraordinario. Compromiso: sin base propia el análisis es fácil de reemplazar, pero un modelo predictivo necesitará historia para entrenar. Riesgos: R-05.	reemplazar el cálculo por reglas por una segunda implementación simulada detrás del mismo contrato y verificar 0 cambios en Inventario, Distribución y Catálogo.

6.2 Riesgos

ID	Riesgo	Impacto	Mitigación	Estado
R-01	Si en bodega se registra tarde, de memoria, o no se saca el lote que FEFO asignó, el sistema no refleja lo que pasó y la trazabilidad se pierde aunque la arquitectura la soporte.	Alto: invalida QA-01 en la práctica.	Pantallas para celular con pocos campos (ADR-016); la remisión impresa dice qué lote sacar; las excepciones se registran como baja o ajuste; probar con el personal.	Abierto
R-02	Un usuario administrador de la base o un script de migración puede modificar movimientos aunque la aplicación no pueda.	Medio	La aplicación usa un usuario sin UPDATE ni DELETE sobre movimientos; las migraciones no tocan datos de movimientos.	Abierto
R-03	Si el aviso al análisis se espera dentro de la petición, la falla del análisis bloquea la recepción.	Alto: incumple QA-04.	Evento interno después del commit y HTTP sin espera con timeout corto (ADR-003, ADR-017).	Mitigado
R-04	Un cambio de vida útil también cambia la fecha estimada de los lotes ya recibidos, así que una fecha vista ayer puede ser distinta hoy.	Bajo: los cálculos de riesgo anteriores quedan guardados en RiesgoProducto.	Decidido para el MVP (2026-10-17): la fecha estimada se calcula al consultar (recepción + vida útil vigente) y no se guarda en el lote. Conservar la fecha original de cada lote queda como mejora futura.	Aceptado
R-05	El modelo predictivo necesitará datos (histórico, donante, temporada) que el contrato actual no expone y que hoy no existen (AS-3).	Medio	Versionar el contrato del análisis; prever una consulta de histórico de movimientos para entrenamiento.	Abierto
R-06	El destino y el formato del reporte mensual (¿ABACO?) no están confirmados, ni cómo se cuenta la población en entregas directas a personas.	Medio	Formato como adaptador de salida (Clean Architecture); preguntas abiertas al banco.	Aceptado como pendiente
R-07	Dos transacciones que bloquean lotes en distinto orden se esperan mutuamente (deadlock).	Medio	Bloquear siempre los lotes en el mismo orden (FEFO, luego Id) y reintentar una vez ante deadlock.	Mitigado: táctica de ADR-013
R-08	Un módulo que abra su propia conexión queda fuera de la transacción compartida y deja datos a medias.	Alto: incumple QA-03 y QA-05.	Transacción compartida en Ssd.Shared; pruebas de integración que fallan si un módulo confirma por su cuenta.	Mitigado: Unit of Work y ADR-017

ID	Riesgo	Impacto	Mitigación	Estado
R-09	Al habilitar un segundo banco, una conexión reutilizada puede conservar el esquema de otro banco, o el administrador puede enviar un banco que no le corresponde.	Alto, pero solo con más de un banco (AS-5).	Fijar el esquema en cada transacción; validar X-Banco-Id contra el rol y auditar; pruebas de aislamiento.	Aceptado: pospuesto (ADR-011)
R-10	El costo fijo mensual puede superar lo que el banco puede pagar.	Medio	Estimar con la calculadora de AWS y fijar un tope; apagar el entorno fuera de horario durante el prototipo.	Aceptado como pendiente
R-11	Si cae la zona de disponibilidad o la tarea del monolito, todo el sistema queda fuera de servicio.	Medio	Aceptado para el prototipo (CON-1); Multi-AZ si pasa a producción.	Aceptado
R-12	Sin internet en la bodega no se puede registrar la recepción (AS-2).	Alto si AS-2 es falso.	Validar AS-2 con el banco; un modo sin conexión queda como evolución futura.	Aceptado: fuera de alcance
R-13	Reglas del dominio por confirmar: reempaque de presentaciones, población de entregas directas, vida útil sin fecha, desechado pesado o estimado, pedido de lote específico (AS-7) y armado de kits (AS-8).	Medio: pueden cambiar el modelo de datos.	Preguntas abiertas al banco; el modelo actual no las bloquea.	Abierto
R-14	La atención de consultas, rectificación y supresión de los titulares (Ley 1581) no está en el MVP.	Medio si el prototipo usa datos reales.	Usar datos de prueba en el prototipo; si se incorpora, anonimizar al titular conservando la traza.	Aceptado (2026-10-16)
R-15	Un peso por presentación mal registrado distorsiona los kg del reporte, aunque las existencias en su unidad estén bien.	Medio	Peso obligatorio y mayor que cero (ADR-012); el coordinador revisa el catálogo antes del primer reporte.	Mitigado parcialmente
R-16	Amazon SES empieza en modo sandbox y solo envía a correos verificados.	Medio: las invitaciones no llegan.	Verificar el dominio con DKIM en Route 53 y pedir salir del sandbox antes de invitar usuarios reales.	Abierto: tarea operativa
R-17	Cualquier cuenta de Google puede autenticarse en Cognito aunque no tenga invitación.	Bajo: el backend le niega el acceso.	Aceptado; una Lambda pre sign-up queda como mejora si los registros sobrantes molestan.	Aceptado

6.3 Temas de Riesgo

- La trazabilidad depende de la disciplina en bodega (R-01, R-12): la arquitectura la hace posible, pero la operación decide si es real.
- La integridad depende de una sola transacción bien usada (R-07, R-08): es la pieza técnica más delicada y se cubre con Ssd.Shared y pruebas de integración.

- La disponibilidad completa es limitada a propósito (R-11): QA-04 aísla las fallas del análisis, no las del monolito ni de la zona; aceptable para un prototipo.
- Hay incertidumbre del dominio y del entorno (R-06, R-10, R-13, R-16): reglas, reporte, costo y correo dependen de confirmaciones del banco o de tareas operativas.